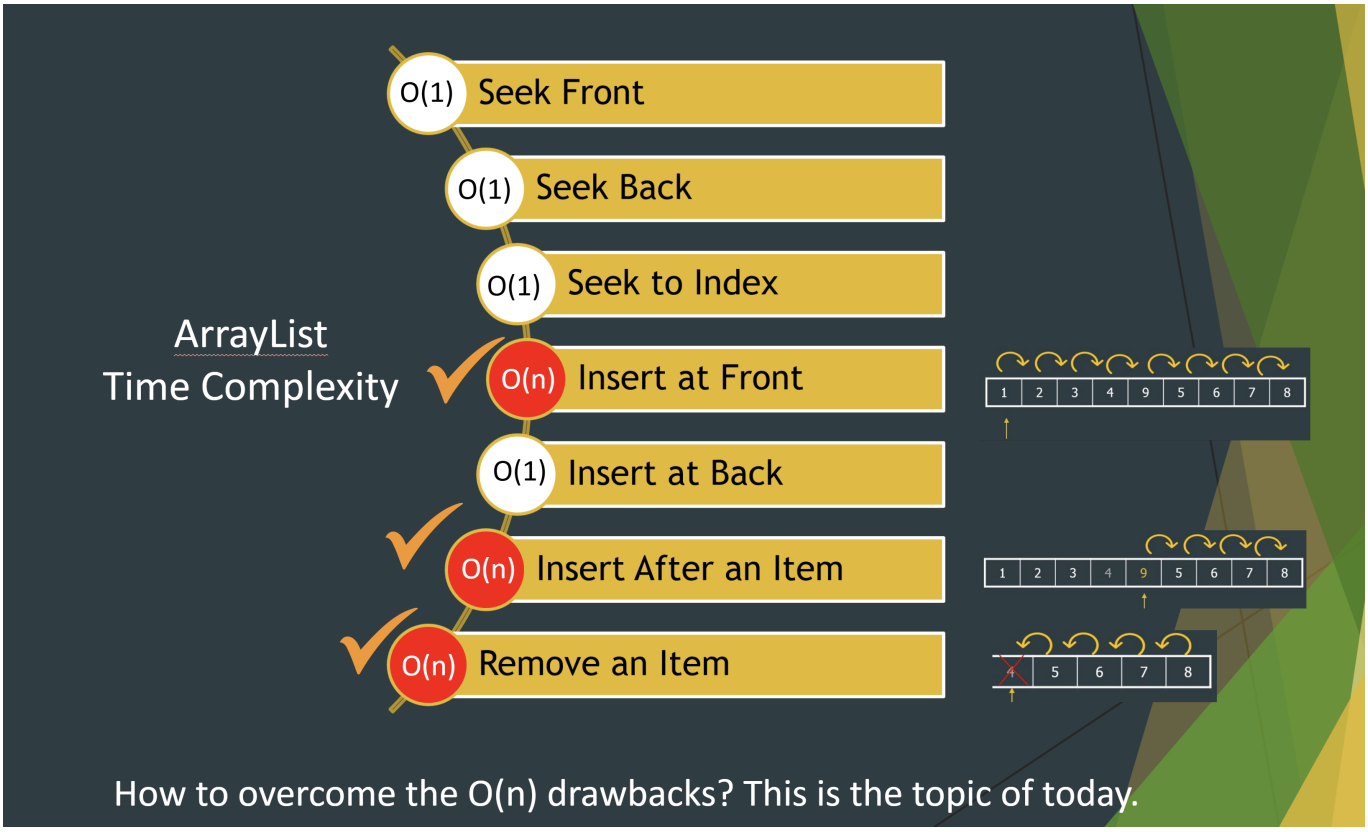
LinkedList & ArrayList Review

Both total O(n)

- ArrayList
  - Search O(1)
  - Shift O(n)
- LinkedList
  - Search O(n)
  - Remove O(1)

When removing an object, make sure to use the `.remove()` method as it returns a boolean to check if it actually gets removed.

ArrayList



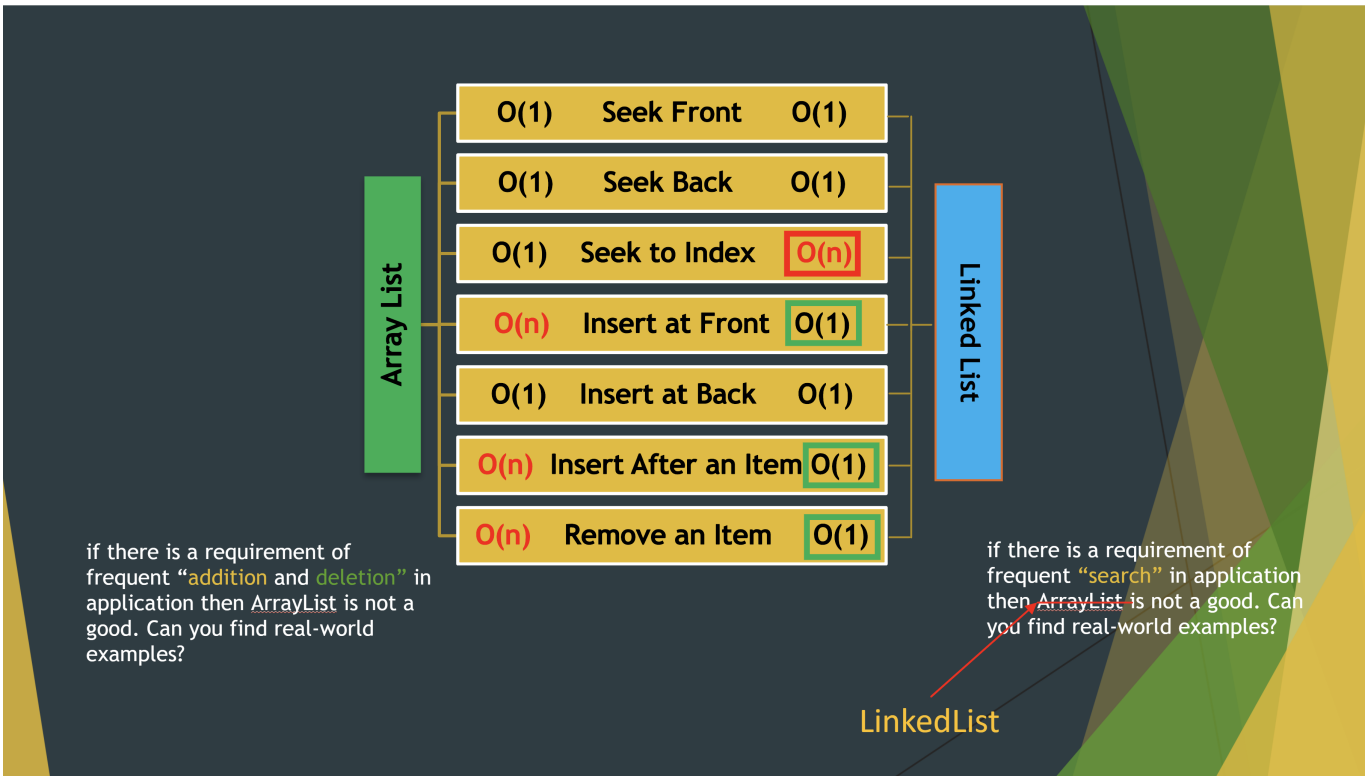
> answer is LinkedList. Removing only requires altering one node. Searching drawbacks though.

```
ArrayList<Integer> al = new ArrayList<>();
al.add(1);
al.remove(0); //index 0, if u want to remove integer 1. just do remove((Object)
1).
```

Methods	Return
`add()`	`E`
`get(int index)`	`E`
`remove(int index)`	`E`
`remove(Object o)`	`boolean`

LinkedList

LinkedLists work by having each node contain one value and point to the next one. It allows for very fast removal and insertion as you only need to change one node. However, you sacrifice searching time as you have to traverse through each node. In theory, you have a head node and a `.data()` attribute. You can do `.next` to go the next node. You can't access these internal nodes with the `java.util.LinkedList` program and can only do the basic operations.




```
LinkedList<String> list=new LinkedList<String>();

//Adding elements to the Linked list
list.add("CS210"); list.add("CS211");
list.add("CS212"); list.add("CS213");
list.add("CS214");

int i=list.indexOf("CS311");
System.out.println("index of CS311 : "+i);

int j=list.indexOf("CS211");
System.out.println("index of CS211 : "+j);
```

LinkedList.indexOf(Object element) method is used to check and find the occurrence of a particular element in the list. If the element is present then the index of the first occurrence of the element is returned otherwise -1 is returned if the list does not contain the element.

Methods	Return
`remove()`	`E`
`remove(int index)`	`E`
`remove(Object o)`	`boolean`
`push(E item)`	`E`
`search(Object o)`	`int`